

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Tecnicatura Universitaria en Inteligencia Artificial



Aprendizaje Automático II

IA 5.1

Informe N°3

Año: 2026

Grupo: 3

Integrantes:

- Calabozo, Nicolas - C-7551/5
- Armas, Alejandro - A-4685/1
- Romano, Maximiliano - R-4634/5

Fecha de presentación: 26/06/2026

Docentes:

- Leandro Salvaña
- Florencia Fernández

Parte I — Q-Learning Clásico

Ejercicio 1: Taxi-v3

Para la resolución de esta etapa, se utilizó el entorno Taxi-v4 (debido a que la versión v3 ya no se encuentra en funcionamiento). El MDP abordado se define a través de la siguiente tupla (S , A , P , R , γ):

- **Espacio de Estados (S):** Es un espacio discreto compuesto por 500 estados posibles. Este valor surge de las combinaciones geométricas y lógicas del entorno: 25 posiciones del taxi (grilla de 5×5), 5 ubicaciones posibles del pasajero (4 paradas específicas + 1 a bordo del taxi) y 4 destinos posibles ($25 \times 5 \times 4 = 500$).
- **Espacio de Acciones (A):** Discreto, con 6 acciones deterministas: moverse al Sur (0), Norte (1), Este (2), Oeste (3), recoger pasajero (*pick-up*, 4) y dejar pasajero (*drop-off*, 5). Esto da como resultado una Q-Table de tamaño $S \times A$, resultando en 3000 entradas.
- **Función de Recompensa por Defecto (R):** El entorno base provee una señal de recompensa dispersa. Emite una magnitud de +20 únicamente al dejar al pasajero en el destino correcto. Adicionalmente, emite penalizaciones de -1 por cada paso temporal (incentivando rutas cortas) y de -10 al ejecutar *pick-up* o *drop-off* de forma ilegal.
- **Función de Transición (P):** En la configuración estándar de este entorno, las transiciones son deterministas ($P(s'|s,a) = 1$). Es decir, no existe estocasticidad (como resbalones u obstáculos móviles): si el agente decide moverse en una dirección libre, avanzará con un 100% de certeza. Si decide moverse contra una pared, permanecerá en el mismo estado con un 100% de certeza.
- **Factor de descuento (γ):** Se eligió un valor cercano a 1. Al tratarse de una tarea episódica donde la única recompensa positiva significativa se encuentra estrictamente al final de la secuencia de acciones, es necesario que el agente valore fuertemente las recompensas futuras a largo plazo para que la ecuación de Bellman logre propagar el +20 hacia los estados iniciales.

Para la resolución del problema se implementó un algoritmo de Q-Learning desde cero. Primeramente, se inicializó la Q-Table en ceros, puesto que el agente todavía no ha comenzado a aprender. Junto a esto, se utilizó una política ϵ -greedy con decaimiento exponencial para lograr un balance entre exploración y explotación. Esto permite que, al comienzo, el agente tenga un ϵ grande para explorar libremente los estados del problema mediante acciones aleatorias. Luego, gracias al decaimiento exponencial, el valor de ϵ disminuye gradualmente, reduciendo las posibilidades de explorar al azar y dando paso a una explotación cada vez más agresiva para conseguir las mayores recompensas esperadas.

La mayor dificultad en el trabajo fue diseñar e implementar dos funciones de recompensa alternativas a la que provee el entorno por defecto.

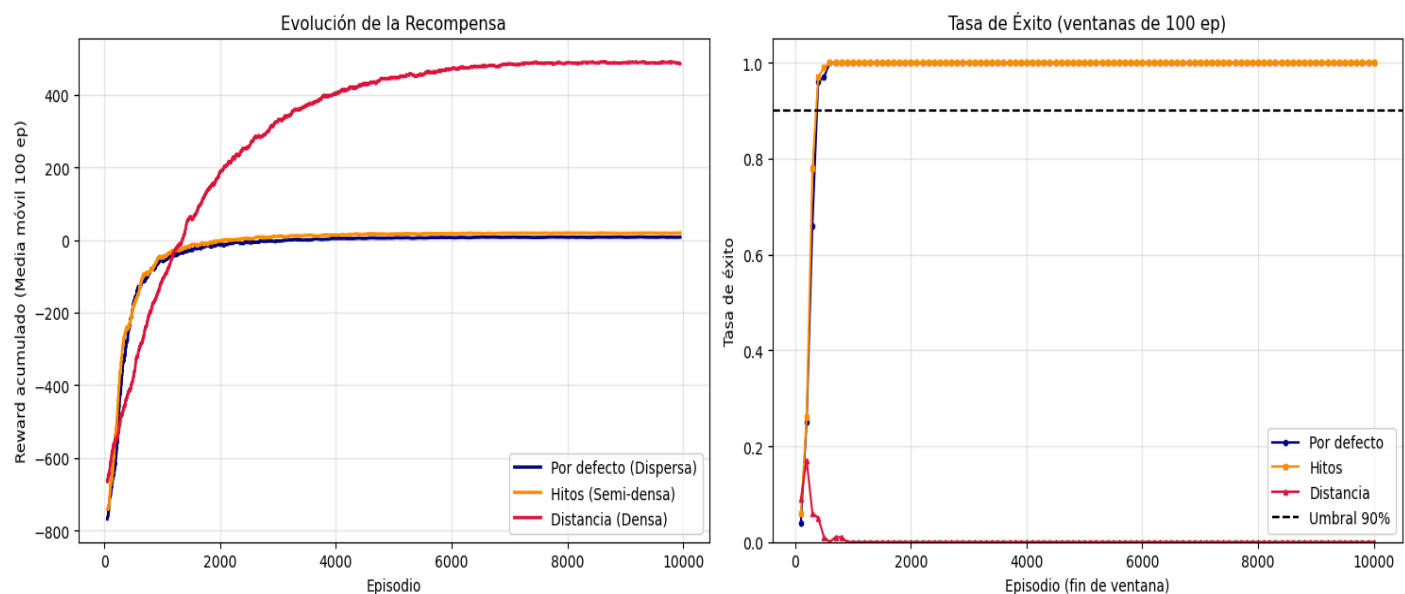
Para la primera función se optó por una que utilizara la Distancia Manhattan entre la posición del taxi y el objetivo actual. La idea detrás de esto es premiar al agente al hacer movimientos que reduzcan la distancia al objetivo y penalizar los que hagan que se aleje. De este modo, el agente recibe una retroalimentación inmediata ante cada transición. También se premia el subir y bajar al pasajero cuando corresponde y se penaliza el hacerlo de forma ilegal. El esquema de recompensas definido para la función es:

- +50 al completar con éxito el episodio (dejar al pasajero en el drop-off correcto).
- +15 como subobjetivo al recoger al pasajero en el pick-up correcto.
- +1 por cada paso que disminuya la Distancia Manhattan hacia el objetivo actual.
- -1 por cada paso que no disminuya la distancia (ej. chocar contra una pared).
- -2 por cada paso que aumente la Distancia Manhattan.
- -10 al ejecutar las acciones de pick-up o drop-off de forma ilegal o en casilleros incorrectos.

Para la segunda función se definió una alternativa basada en hitos intermedios. Se conservan las penalizaciones y la recompensa de la función original, y se agrega una nueva recompensa al subir al pasajero en el lugar correcto. Al hacer esto se busca dividir el problema en dos, otorgando una recompensa para que el agente tenga una confirmación temprana sobre si está actuando correctamente, sin tener que esperar a terminar el trayecto para saberlo. El esquema de recompensas definido es el siguiente:

- +20 al completar con éxito el episodio (dejar al pasajero en el drop-off correcto).
- +10 como premio intermedio al recoger al pasajero en el pick-up correcto.
- -1 por cada acción de movimiento (arriba, abajo, izquierda, derecha), incentivando la resolución rápida del problema.
- -10 al ejecutar las acciones de pick-up o drop-off de forma ilegal o en casilleros incorrectos.

Comparación de Convergencia — Funciones de Recompensa en Taxi-v4



A partir de las gráficas de entrenamiento, se puede observar que la función de recompensa basada en hitos es la que converge más rápido y presenta mayor estabilidad. Esta leve superioridad frente a la función por defecto se debe a que el premio otorgado durante el transcurso del episodio actúa como una señal confirmatoria temprana. Le indica al agente que está resolviendo bien la primera mitad del problema, mitigando la dificultad de propagar el valor de la meta final a través de trayectorias largas. Utilizando la función de hitos, el agente logró alcanzar el criterio de corte de una tasa de éxito del 90% en 350 episodios, en comparación con los 357 episodios que requirió la función por defecto.

Por otro lado, la función basada en la Distancia Manhattan resultó ser deficiente, obteniendo recompensas acumuladas fuertemente negativas y nula tasa de éxito. Esto se explica por la topología del entorno (paredes internas). Al intentar rodear una pared, el agente está obligado a moverse en una dirección que lo aleja temporalmente de su objetivo (incrementando la distancia geométrica). Al penalizar estrictamente este alejamiento, el agente queda atrapado en un mínimo local: descubre que chocar contra la pared (o quedarse estancado) le otorga una penalización menor que la de intentar rodear el obstáculo. Como consecuencia, el agente se estanca y agota el límite temporal del episodio.

El desarrollo del algoritmo tabular demostró la capacidad de Q-Learning para encontrar políticas óptimas en espacios de estados finitos. La principal lección de esta etapa recae en las limitaciones del *Reward Shaping*. Modificar artificialmente las recompensas para acelerar la convergencia (como el caso de la Distancia Manhattan) puede resultar contraproducente e inducir comportamientos subóptimos o estancamientos si la función de recompensa no refleja las restricciones físicas y lógicas del entorno real. En contraste, el enfoque por hitos demostró que inyectar conocimiento del dominio sin alterar las mecánicas de exploración originales resulta ser el método más seguro y robusto para optimizar el aprendizaje de un agente.

Parte II — Deep Q-Learning (DQN)

Ejercicio 2: LunarLander-v3

LunarLander-v3 simula el aterrizaje de una nave espacial en una plataforma. El agente controla tres propulsores y debe aterrizar suavemente entre las banderas que marcan la zona de destino.

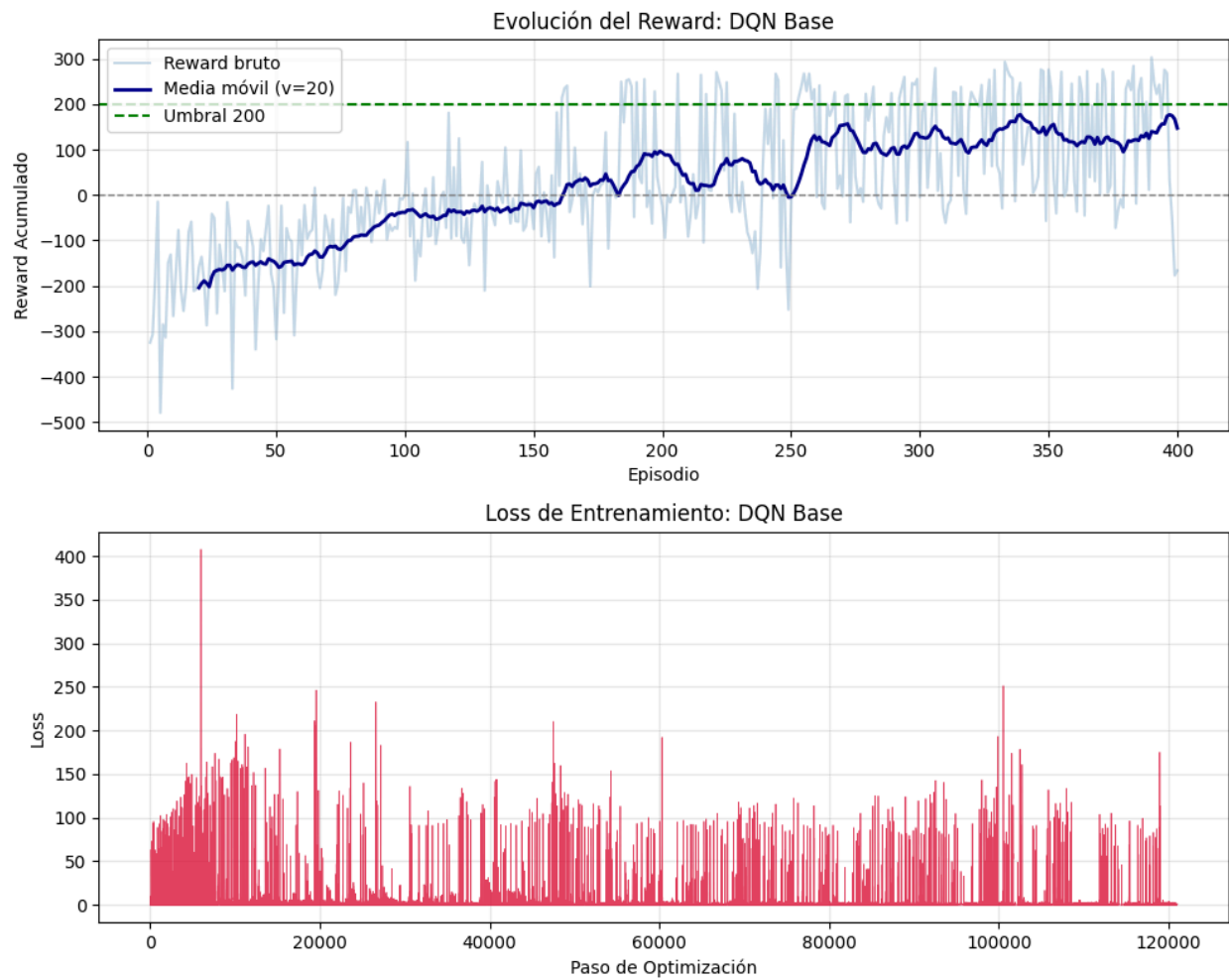
Análisis del MDP

- **Espacio de estados (S):** El estado de la nave está dado por un vector continuo con las siguientes 8 componentes:
 - Sus coordenadas cartesianas (X,Y)
 - Su velocidad inicial (Vx, Vy)
 - Su ángulo respecto al eje horizontal (θ)
 - Su velocidad angular (ω)
 - 2 Valores booleanos representando la colisión de una de las dos patas con el suelo
- **Espacio de acciones (A):** La nave tiene 3 motores propulsores: inferior, izquierdo y derecho. Los cuales solo pueden encenderse a máxima potencia, uno a la vez (Acciones discretas). Las acciones posibles entonces son:
 - [0] No hacer nada
 - [1] Disparar propulsión izquierda
 - [2] Disparar propulsión inferior
 - [3] Disparar propulsión derecha
- **Función de recompensa (R):** Durante cada episodio, la recompensa es acumulativa y se calcula en cada paso/frame, siguiendo las siguientes reglas:
 - Aumenta/disminuye según que tan cerca/lejos esté la nave de la plataforma de aterrizaje
 - Aumenta/disminuye según que tan lento/rápido se esté moviendo la nave
 - Disminuye según que tan inclinada esté la nave (ángulo no horizontal)
 - Aumenta 10 puntos por cada pata que esté en contacto con el suelo
 - Disminuye 0.03 puntos cada frame que un motor lateral esté encendido
 - Disminuye 0.3 puntos cada frame que el motor inferior esté encendido
- **Función de Transición (P):** Es un entorno determinístico, cada estado (excepto el inicial) está completamente determinado por el estado anterior y la acción tomada. ($P(s'|s,a) = 1$)
- **Factor de descuento (γ):** 0.99, Valor alto para que el agente explore estados con recompensas a largo plazo, ya que aterrizar correctamente representa la mayor ganancia de puntos y es una recompensa a largo plazo dentro de cada episodio.

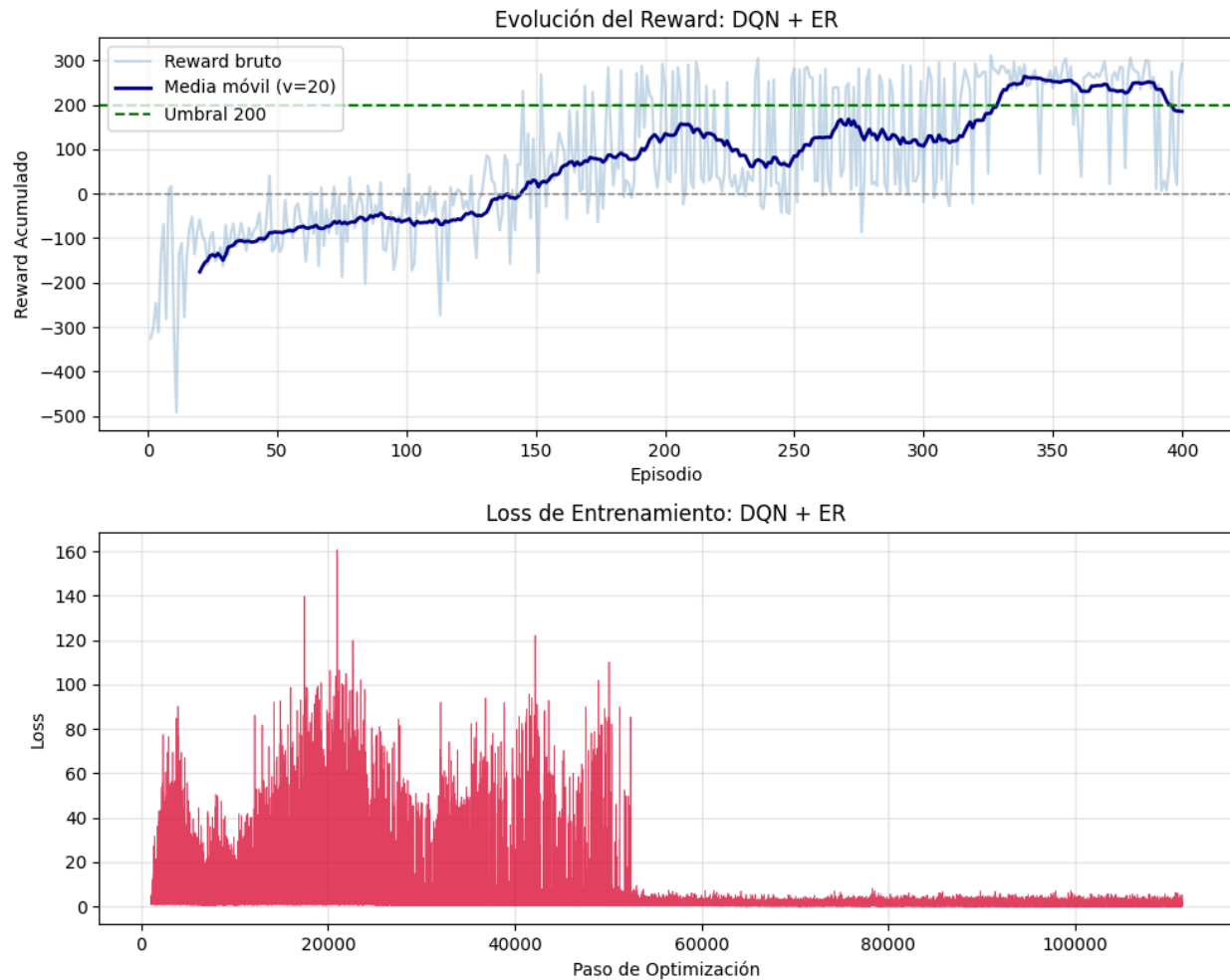
Se implementaron 3 versiones de DQN, para poder comparar el impacto de cada innovación individual (*Experience Replay* y *Target Network*)

Comparación de versiones

(a) DQN base



(b) DQN + Experience Replay

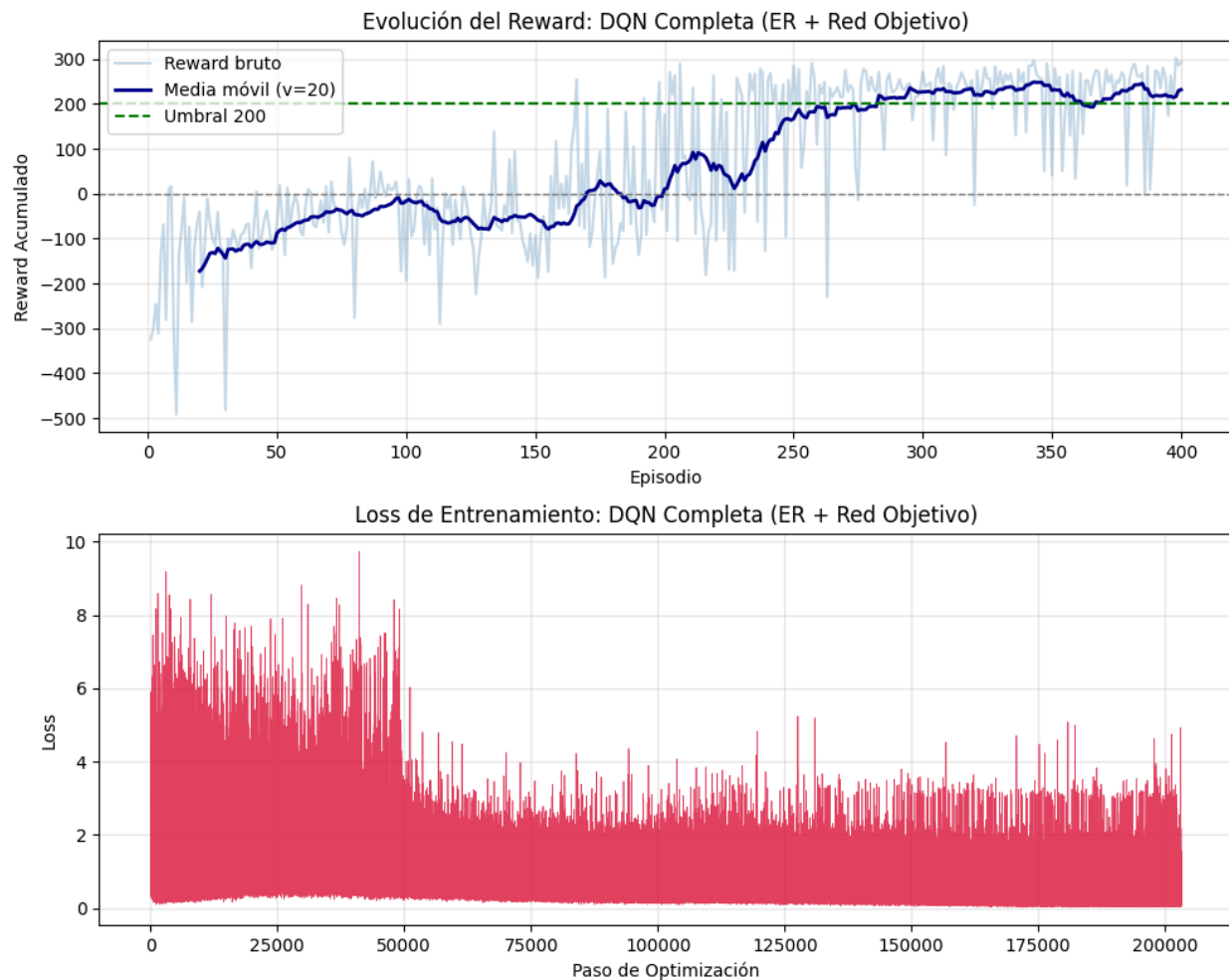


Observamos que la incorporación de *Experience Replay* mejora absolutamente el desempeño de la DQN, obteniendo recompensas más altas y convergiendo a valores de loss muy inferiores.

Esto se debe a que en la versión (a), cada muestra estaba completamente relacionada con la anterior (siguiente frame de la simulación), violando el supuesto de muestras *i.i.d* necesario para el descenso por el gradiente, lo que produce divergencia puesto que la red siempre está ajustando los pesos secuencialmente (siguiendo rachas de datos correlacionados) y no aprende a generalizar.

El Experience Replay que incorpora (b) soluciona el problema llevando una cola de transiciones de las cuales toma una muestra aleatoria (batch) para cada paso de optimización, lo que produce que la red aprenda de situaciones distintas en cada paso, aprendiendo a generalizar correctamente.

(c) DQN + Experience Replay + Target Network



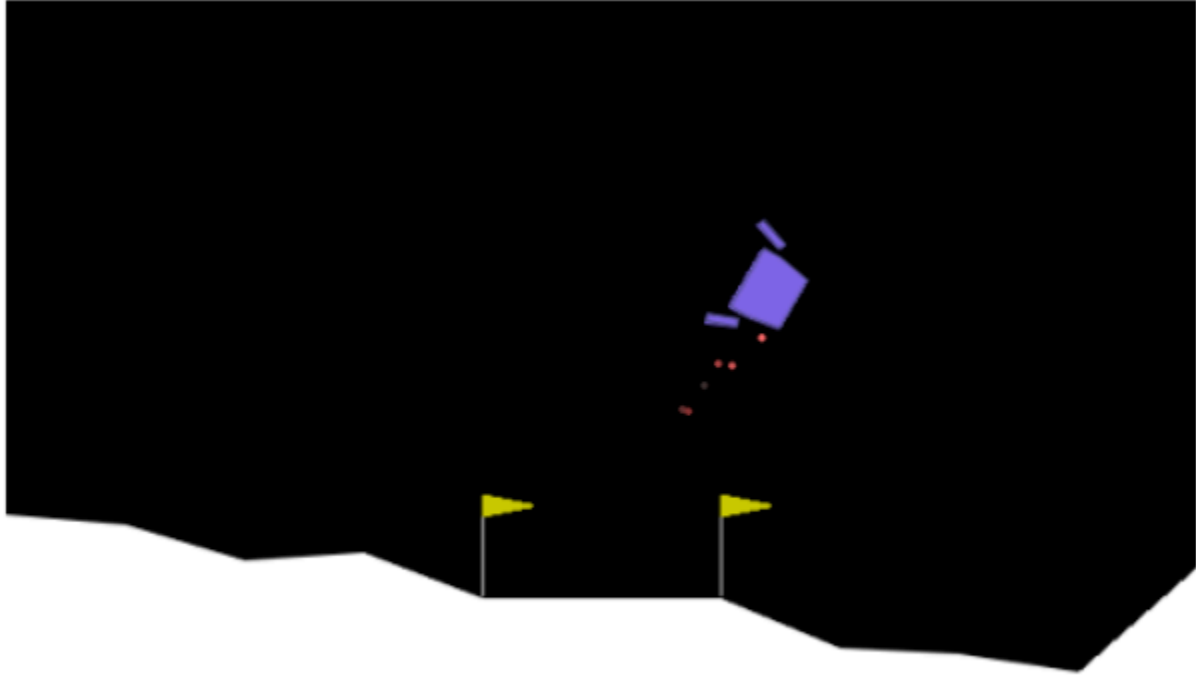
Los problemas de moving target se manifiestan en las versiones (a) y (b), que se evidencia por las curvas de loss más erráticas y oscilantes, visible durante todo el entrenamiento de (a) y también durante la primera mitad del entrenamiento de (b), el cuál sin embargo termina por converger y estabilizar su loss en valores bajos, pero tomando más episodios que (c).

Esto se debe a que tanto (a) como (b) cambian sus valores óptimos en cada paso de optimización, ya que usan la misma red neuronal tanto para decidir la acción como para calcular el target. En cambio (c) utiliza una segunda red congelada (*Target Network*), cuyo estado se copia desde la red principal (policy) cada cierta cantidad de episodios, permitiendo que la red policy pueda “perseguir un mismo objetivo” durante varios episodios.

La incorporación de la Target Network sobre la versión (b) se traduce en una mayor velocidad de aprendizaje, y también un aprendizaje más profundo que le permite alcanzar rewards más altos. Sin embargo destacamos que el cambio de incorporar *Experience Replay* -de (a) a (b)- fue mucho más notorio que el de incorporar Target Network -de (b) a (c)- .

Comparación de episodios durante distintos momentos del entrenamiento del agente

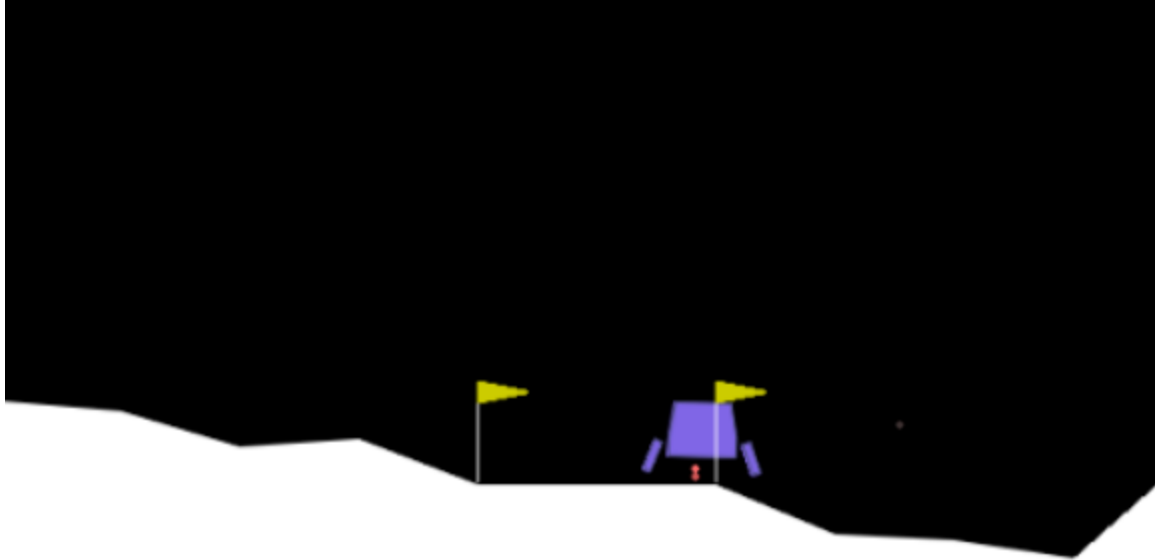
(Animaciones en notebook)



En el primer episodio (sin entrenar) observamos que la nave rota sobre sí misma y simplemente se cae hacia la derecha, si bien empieza a usar el propulsor derecho (que ayudaría a estabilizar la nave) lo hace tarde y termina por chocar con la luna.

Luego de 200 episodios, la nave se mantiene estable y paralela al eje horizontal (usando sus propulsores laterales) y usa su propulsor principal constantemente para amortiguar la gravedad y moverse lento. Este aprendizaje se debe a que mantenerse horizontal y propulsándose desde abajo son las mejores estrategias para **no chocar**, que es lo primero que aprende ya que el descuento en reward por chocar es muy grande (-100).

Además, el hecho de no estar alineado con la horizontal y de moverse velozmente (no amortiguar gravedad) también produce decrementos en el reward.



Una vez completado el entrenamiento (alcanzado un reward medio ≥ 200 en 100 episodios consecutivos en el episodio 326), la nave aterriza correctamente dentro de la zona indicada. Como su posición inicial siempre está encima de la zona de aterrizaje, aprende que una buena estrategia es simplemente intentar corregir su velocidad inicial aleatoria para mantenerse estable y usar el propulsor inferior para amortiguar la caída.

Parte III — Análisis teórico comparativo

Ejercicio 3

Comparación de métodos

	Taxi-v4 Q-Learning	LunarLander-v2 DQN
Tipo de espacio de estados	Finito y discreto.	Continuo.
Representación de la política aprendida	Q-Table.	Red neuronal (8 -> 128 -> 128 -> 4)
Generalización entre estados similares	No posee capacidad de generalización.	Si, la red comparte representaciones internas.
Episodios hasta convergencia	Según la función de recompensa aproximadamente: 2000, 6000 y 2000.	Entre 280 y 330
Tiempo de entrenamiento aproximado	Según la función de recompensa: 9.8 s, 105.4 s y 11.2 s.	150s

Síntesis:

Q-Learning clásico es preferible cuando el espacio de estados es discreto y de tamaño manejable como Taxi-v4 con 500 estados, porque ofrece una solución simple e interpretable. DQN es preferible cuando el espacio de estados es continuo o demasiado grande para discretizar sin que la tabla resultante sea inviable, aceptando a cambio un mayor costo computacional por actualización.

El rol del factor γ

- En un entorno como Taxi-v4, la dinámica presenta un castigo constante de -1 por cada paso y un premio de +20 que es disperso y sólo se descubre en el instante final. En este contexto, implementar un factor de descuento γ cercano a 1 permite que el valor de la recompensa final se propague hacia atrás mediante la ecuación de Bellman, sobreviviendo a través de las múltiples transiciones. Esto garantiza que los estados

tempranos del episodio perciban la recompensa final, guiando al agente desde el primer movimiento y evitando que desarrolle un comportamiento miope.

- b) En LunarLander-v3 la duración de los episodios no es homogénea a lo largo del entrenamiento. Al inicio, el agente estrella la nave en pocos pasos, generando episodios cortos donde γ prácticamente no influye, en esta fase el valor de γ es casi irrelevante. A medida que el agente mejora, los episodios se alargan considerablemente, la nave maniobra durante muchos más pasos antes de aterrizar, y las recompensas positivas por contacto suave con el suelo quedan cada vez más lejos en el tiempo. Aca γ se vuelve un factor crítico, como observamos en el experimento A donde disminuimos γ a 0.98, la convergencia tardó mucho más (episodio 740 contra episodio 326), un valor aún más bajo haría que esas recompensas tardías lleguen excesivamente atenuadas a los estados tempranos, dificultando que el agente aprenda a planificar la trayectoria de descenso completa.

Reflexión final

- a) Se plantea un nuevo problema de RL con un espacio de estados discreto y finito de tamaño 10.000 y 8 acciones posibles. ¿Utilizarías Q-Learning clásico o DQN?

Para un nuevo problema con un espacio discreto y finito de 10.000 estados y 8 acciones, se generaría una tabla de valores con 80.000 entradas posibles (10.000 estados $\times$ 8 acciones). Esta cantidad de datos es pequeña y fácil de procesar por lo que la elección recomendada sería utilizar Q-Learning clásico. El algoritmo tabular es mucho más sencillo de implementar y menos costoso a nivel de hardware. La aproximación por DQN está diseñada para espacios de estados masivos o continuos.

¿De qué otros factores dependería la decisión más allá del tamaño del espacio de estados?

La decisión dependería además de otros factores. La capacidad de generalización entre estados, si los 10.000 estados no guardan ninguna relación semántica entre sí, el enfoque tabular es mejor opción, ya que DQN no tendría estructura subyacente que explotar y su ventaja de generalización no aportaría nada. Pero si los estados comparten características subyacentes, una red neuronal podría generalizar lo aprendido en estados visitados a otros similares no explorados, acelerando el aprendizaje. Si la señal es muy dispersa, Q-Learning puede tardar mas en propagar el valor a través de los 10.000 estados, y en ese caso DQN tiene mas sentido independientemente del tamaño del espacio.

- b) La limitación más importante que queda sin resolver en LunarLander-v3 es la ineficiencia muestral, el agente requiere una cantidad muy grande de experiencia real para que la red neuronal ajustar sus pesos correctamente, especialmente en las etapas iniciales donde la señal de recompensa es casi siempre negativa y poco informativa.

Los algoritmos que abordan esta limitación son las extensiones de DQN, Prioritized Experience Replay, que en lugar de samplear transiciones de forma uniforme prioriza aquellas con mayor TD error, las mas informativas, logrando aprender más de cada interacción y Double DQN que reduce la sobreestimación sistemática de los valores Q, haciendo que cada actualización sea más precisa y que el agente necesite menos muestras para corregir sus estimaciones.

La idea central detrás de ambos es la misma, extraer más información útil de cada experiencia recolectada.